Agentic RAG on Oracle Cloud — The 4-Day Journey

Three production-style agentic-RAG prototypes, built on Oracle Cloud Infrastructure (OCI) with LangGraph. Every prototype ships with human-in-the-loop control and guardrails. Half taught, half built.

OCI Generative AI | Oracle DB 23ai Vector Search | LangGraph | Human-in-the-loop | Guardrails | 4 days · 50% taught · 50% built

1 What we are running

- **Format:** a four-day AI-engineering hackathon.
  - **Days 1–3:** teams build working prototypes.
  - **Day 4:** teams present them live.
- **Platform:** everything runs on Oracle Cloud Infrastructure (OCI) and is orchestrated with LangGraph.
- **Project family:** all three prototypes are **agentic RAG** systems.
  - **RAG** — the system answers a question by first *retrieving* relevant information, then *generating* an answer grounded in what it found.
  - **Agentic** — the system does not answer in one blind shot. It:
    - checks its own work,
    - retries when the retrieval was weak,
    - pauses for a human when a decision matters,
    - refuses when a guardrail is tripped.

MANDATORY

Human-in-the-loop (HITL)

The agent pauses and waits for a person to approve, edit, or reject before it does anything consequential.

MANDATORY

Guardrails

Automatic checks on what goes in and what comes out, so the agent stays safe, on-topic, and grounded.

OPTIONAL

Memory

Short conversational memory for follow-up questions. A bonus, never a requirement.

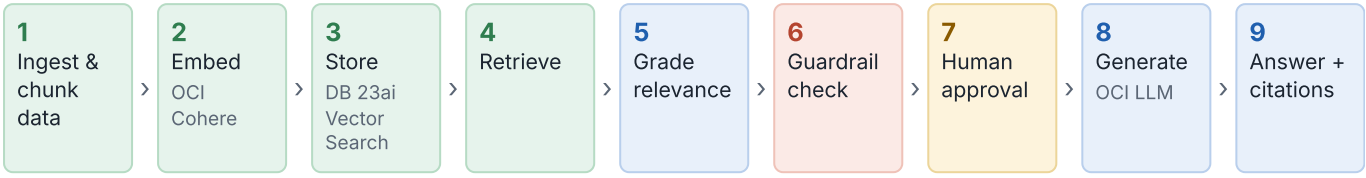
2 The three prototypes at a glance

#	Prototype	Talks to...	The "agentic RAG" idea	Signature HITL / guardrail
A	**Talk to your Database** NL2SQL assistant	An Oracle database	Retrieves the right tables, columns and business glossary to *ground* the SQL it writes.	• Human approves the SQL before it runs. • Guardrail blocks any write/drop and enforces read-only.
B	**Ask your Reports** PDF + Excel assistant	PDF reports and Excel files	Retrieves relevant passages and table rows, grades them, and re-retrieves if weak.	• Agent asks a human to clarify ambiguous questions. • Guardrail refuses any answer without a cited source.
C	**Interrogate your Logs** Service-log assistant	Server / application logs	Retrieves the relevant log windows (by time and service) and reasons over them for root cause.	• Human approves any suggested remediation. • Guardrail redacts secrets on the way in and forbids fabricated log lines.

Each prototype has its own detailed document (Documents 1–3). They share the same backbone, so once a team understands one, the next two feel familiar.

3 The shared backbone every prototype uses

All three prototypes are the same nine-step pipeline. Only the *data* and a few stage details change.



☐ Retrieval ☐ Reasoning ☐ Guardrail ☐ Human-in-the-loop ☐ Optional memory

Loop: step 5 → step 4 (rewrite the query and retry when retrieval is weak) **Side-store:** optional memory of earlier turns

THE PIPELINE IN PLAIN WORDS

- **Steps 1–3:** turn the data into searchable "chunks" and store them.
- **Step 4:** find the chunks most relevant to the question.
- **Step 5:** check whether they are good enough; if not, retry.
- **Step 6:** run the safety checks.
- **Step 7:** pause for a human when it matters.
- **Steps 8–9:** write a grounded answer and show its sources.

4 How the 50 / 50 model works

Each build day follows the same rhythm. The facilitator *teaches by building* one part of the pipeline live; teams then rebuild and extend that same part on their own prototype. Nobody watches a finished solution — they watch a stage being built, then build it themselves.

Slot	What happens	Who leads
Morning block	Facilitator builds one pipeline stage live, explaining the concept and the OCI connection as they go.	TAUGHT
Afternoon block	Teams build the same stage on their assigned prototype, then push it further using the stretch tasks.	BUILT
End of day	Every team must pass that day's **checkpoint** — a concrete, verifiable "it works" test.	Both

5 The day-by-day plan

Day 1 — Foundation and the baseline RAG spine

Goal: get from raw data to a first grounded answer. No agentic behaviour yet — just a clean pipeline that retrieves and generates.

- TAUGHT Wire up OCI:
  - authentication,
  - the chat model and the embedding model,
  - the Oracle 23ai vector store,
  - an introduction to the LangGraph state machine.
- TAUGHT Build stages 1–4 and 8–9: ingest → embed → store → retrieve → generate.
- BUILT Teams reproduce the spine on their own prototype's data.

DAY-1 CHECKPOINT
Ask one real question and get a coherent answer that is grounded in retrieved content, with the sources printed alongside it.

Day 2 — Make it agentic, make it safe

Goal: turn the spine into an agent. Add self-correction, guardrails, and the human-in-the-loop pause.

- TAUGHT Relevance grading and the rewrite-and-retry loop (stage 5).
- TAUGHT Input and output guardrails (stage 6) — how to block bad input and unsafe output.
- TAUGHT The HITL pause (stage 7) using LangGraph's interrupt-and-resume.
- BUILT Teams add grading, their prototype's specific guardrails, and their HITL pause.

> DAY-2 CHECKPOINT
> The agent visibly:
> - (a) retries after a weak retrieval,
> - (b) blocks at least one unsafe input and one unsafe output,
> - (c) pauses for human approval before its consequential action.

Day 3 — Specialise, harden, and prepare the demo

Goal: finish the prototype-specific logic, optionally add memory, harden the guardrails, and rehearse the demo.

- TAUGHT The one capability unique to each prototype:
  - A — safe SQL execution,
  - B — citation-grounded answers,
  - C — log root-cause reasoning.
- BUILT Teams complete their prototype, add the *optional* memory if time allows, and run a full dry-run of the Day-4 demo.

> DAY-3 CHECKPOINT
> A full end-to-end run, demo-ready: real question → retrieval → guardrail → human approval → grounded answer, with nothing hard-coded.

Day 4 — Presentations and judging

- Each team gives a short **live demo** (not slides only).
- Each team walks through its pipeline.
- Judging uses the rubric below.

6 Demo and judging rubric

Criterion	What judges look for	Weight
It works, live	A real question answered end-to-end, on the day, without fake data.	**25%**
Grounding	Answers are traceable to retrieved sources; no unsupported claims.	**20%**
Human-in-the-loop	The pause is real: a human can approve, edit, or reject, and it changes the outcome.	**20%**
Guardrails	Unsafe input and unsafe output are actually blocked when demonstrated.	**20%**
Engineering clarity	The team can explain their LangGraph graph and every OCI service they used.	**15%**

7 Prerequisites and pre-work (before Day 1)

> WHY PRE-WORK MATTERS
> The biggest risk in a cloud hackathon is losing Day 1 to setup. Every participant should get a working "hello" from an OCI model **before** they arrive.

Access you need

- An **OCI tenancy login**, placed in a **compartment** for the hackathon.
- Access to **OCI Generative AI** in a region where it is available (for example Chicago or Frankfurt) — it is not in every region.

- An **Oracle Database 23ai** (or Autonomous Database) instance with AI Vector Search, reachable from your laptop.
- An **Object Storage bucket** for raw files.
- An **IAM policy** allowing your group to:
  - *use* the generative-ai-family,
  - read the bucket.

Pre-work checklist (do this at home)

#	Step	Done when...
1	Install Python 3.10+ and the packages (below).	`python -c "import langgraph, langchain_oci"` runs clean.
2	Configure the OCI CLI: `oci setup config` writes `~/.oci/config`.	`oci iam region list` returns data.
3	Run the "hello model" snippet (below).	You get a sentence back from an OCI LLM.
4	Connect to Oracle DB 23ai with `python-oracledb`.	`SELECT 1 FROM dual` returns a row.
5	Note your **compartment OCID**, **region endpoint**, and **DB connection string**.	All three are saved where you can copy them.

INSTALL ONCE

```
pip install langgraph langchain-core langchain-oci langchain-oracledb oci python-oracledb pandas
```

HELLO, OCI MODEL — THE ONE SNIPPET EVERYONE RUNS BEFORE DAY 1

```
from langchain_oci import ChatOCIGenAI

llm = ChatOCIGenAI(
    model_id="meta.llama-3.3-70b-instruct",
    service_endpoint="https://inference.generativeai.us-chicago-1.oci.oraclecloud.com",
    compartment_id="ocid1.compartment.oc1..xxxxxxxx",   # your compartment OCID
    model_kwargs={"temperature": 0, "max_tokens": 256},
)
print(llm.invoke("Say hello from Oracle Cloud in one sentence.").content)
```

If this prints a sentence, your authentication, region, and model access are all working — you are ready for Day 1.

Top setup gotchas

- **Wrong region** — Generative AI is only in certain regions. The `service_endpoint` region must be one that hosts it.
- **Missing IAM policy** — "NotAuthorized" almost always means the policy to use generative-ai-family is missing for your group or compartment.
- **Model ID typo** — model IDs change over time. Confirm the exact ID in the OCI console's Generative AI model list.
- **Deprecated import** — use `langchain-oci`, not the old `langchain-community` OCI classes.

8 The shared OCI toolbox

Service	Role in every prototype	How you reach it
OCI Generative AI	• Chat model — reasons and writes answers. • Embedding model — turns text into vectors.	`langchain_oci`: `ChatOCIGenAI`, `OCIGenAIEmbeddings`
Oracle DB 23ai · AI Vector Search	Stores the chunks and their vectors; answers "find the most similar chunks".	`langchain_oracledb`: `OracleVS` + `python-oracledb`

Service	Role in every prototype	How you reach it
Object Storage	Holds the raw files (PDFs, spreadsheets, log dumps) before ingestion.	`oci` SDK: `ObjectStorageClient`
LangGraph	Not an OCI service — the orchestration layer that wires the stages, runs the loops, and provides the HITL pause.	`langgraph`: `StateGraph`, `interrupt`

OPTIONAL EXTRAS (STRETCH GOALS IN DOCUMENTS 1–3)

- **OCI Generative AI rerank** — to sharpen retrieval.
- **OCI Generative AI Agents** (managed service) — used only as a "look what you built by hand" comparison.

9 Plain-English glossary

Term	In one line
Embedding	A list of numbers that captures the meaning of a piece of text, so texts with similar meanings get similar numbers.
Vector search	Finding the stored text whose embedding is closest to the question's embedding.
RAG	Retrieve first, then generate an answer grounded in what was retrieved.
Agentic	The system checks its own work, retries, pauses for humans, and refuses — instead of answering blindly once.
LangGraph	A library for building the agent as a graph of stages ("nodes") with arrows between them ("edges"), including loops and pauses.
State	The shared record the graph passes from stage to stage (the question, the retrieved chunks, the draft, the answer…).
Human-in-the-loop	The graph stops and waits for a person to approve, edit, or reject before continuing.
Guardrail	An automatic check on input or output that can block, redact, or refuse.
Grounding	Every claim in the answer can be traced back to a retrieved source.

HOW TO READ THE REST OF THE SET

Documents 1, 2 and 3 each cover one prototype in full, in the same order: architecture → OCI connection recipes → data and ingestion → pipeline stage by stage → guardrails and human-in-the-loop → optional memory, stretch goals, demo checklist, troubleshooting. Pick any one and the pattern carries across.